

KARADENIZ TECHNICAL UNIVERSITY — GRADUATE SCHOOL OF NATURAL AND APPLIED SCIENCES

Constraint Programming

Modeling Languages I: MiniZinc and Solver Independence

Week 6

Modeling Languages I: MiniZinc and Solver Independence

Assoc. Prof. Tolga Berber

KTU Faculty of Science, Department of Computer Science · https://avesis.ktu.edu.tr/tberber

Learning phases: 1 • Pre-instruction 2 • Content 3 • Participation 4 • Assessment 5 • Follow-through

1 • Pre-instruction Why It Matters?

Every model you have built so far – N-queens, Sudoku, map coloring, the social golfer problem – was also, secretly, a Python program.

- To switch solvers, or hand your model to a colleague who does not use OR-Tools, you would have had to rewrite it entirely
- This week you write the *same kind* of model in a language whose only job is describing it, not running it
- Starting in Unit 3, *which program actually solves it* becomes a one-line decision instead of a rewrite

`model.NewIntVar(...)` and `model.Add(...)` are still Python calls – MiniZinc drops the host program entirely.

This Week

◎ Learning Objectives

By the end of this week, you will be able to:

1. Write a first working MiniZinc model: parameters, decision variables, constraints and a `solve` item
2. Explain what FlatZinc is and read a flattened `.fzn` file
3. Run one unchanged `.mzn` model on Gecode and on a MIP backend, and state what solver independence does and does not guarantee
4. Drive MiniZinc from Python with `Model`/`Instance`/ `Solver.lookup`, and script a multi-solver comparison
5. Build a new model, verify it on Gecode, run it on a MIP backend unchanged, and justify a solver choice

SECTION 1

Unit 1 — Declarative Modeling in MiniZinc

2 • Content Declarative: Stating What, Not How

- A `.mzn` file states *what* a solution must satisfy – never a search loop, a backtracking step or a branching rule
- Contrast Week 1's `model.NewIntVar(0, n-1, "q0")` – still a Python call, executed as an instruction you control

Definition: MiniZinc

A high-level, solver-independent declarative language for expressing constraint satisfaction and optimization models, compiled to FlatZinc before being solved.

Analogy: A Recipe Card, Not a Kitchen's Instructions

Analogy

A recipe card lists ingredients and the finished dish's requirements – "must serve four, must be vegetarian" – without saying which brand of oven cooks it. A MiniZinc model lists variables, domains and constraints without saying which program searches for a solution: the same recipe card works in any kitchen, the same `.mzn` file works with any installed solver.

Declarative does not mean simpler or less expressive – only that the search machinery lives outside the model file entirely.

The Four Building Blocks of a `.mzn` File

- **Parameters** – fixed, known data (`int`, `float`, `bool`, `set of int`, or arrays of these)
- **Decision variables** – unknowns marked `var`, the counterpart of `NewIntVar`/`NewBoolVar`
- **Constraints** – `constraint` items, arbitrary boolean expressions over both
- **`solve` item** – exactly one per model: `solve satisfy;`, `solve minimize expr;` or `solve maximize expr;`

Common Pitfall

Marking every quantity `var` out of five weeks of CP-SAT habit – `capacity` is data, not an unknown; declare it as a plain parameter.

Model and Data: `.mzn` and `.dzn`

- `int: n;` on its own declares a parameter with no value; a companion `.dzn` file supplies `n = 5;`
- Same model/instance split a Sudoku *puzzle* has from the Sudoku *rules* – just explicit as two files

Analogy

A travel-itinerary template stays the same every time you travel; only the dates and city list change from trip to trip – one template, many trips, exactly like one `.mzn` model reused across many `.dzn` data files.

Worked Example: The 0/1 Knapsack, Parameters and Variables

Choose a subset of n items – one `var bool` per item – whose weight stays under `capacity` and whose value is maximized.

```
int: n;
int: capacity;
array[1..n] of int: weight;
array[1..n] of int: value;
```

```
array[1..n] of var bool: x;
```

Four plain declarations match exactly what a `.dzn` file must supply.

Worked Example: The Knapsack's Constraint and Solve

```
constraint sum(i in 1..n)(weight[i] * x[i]) <= capacity;
```

```
solve maximize sum(i in 1..n)(value[i] * x[i]);

output ["selected items: \(x)\n",
      "total value: \(sum(i in 1..n)(value[i] * x[i]))\n"];
```

`weight[i] * x[i]` works because MiniZinc treats `bool` as `0`/`1` in arithmetic – no separate "if selected" branch is needed.

Output: Solving the Knapsack on Gecode

Running `minizinc --solver Gecode knapsack.mzn knapsack.dzn` against $n{=}4$, weights `[2,3,4,5]`, values `[3,4,5,6]`, capacity 7:

Expected Output

```
selected items: [false, true, true, false]
total value: 9
```

Items 2 and 3 (weights 3 and 4, total 7, exactly at capacity), value $4 + 5 = 9$ – a real, solver-verified answer.

3 • Participation Quick Check: Declare and Constrain

For $n{=}4$ items, weights `[2,3,4,5]`, values `[3,4,5,6]`, capacity 7, write the decision-variable declaration and the capacity constraint only.

`array[1..n] of var bool: x;` for the decision variables, and `constraint sum(i in 1..n)(weight[i] * x[i]) <= capacity;` for the capacity constraint.

SECTION 2

Unit 2 — FlatZinc and Model Transformation

2 • Content What Does Your Model Look Like, Compiled?

- You already know N-queens' variables, domains and constraints on paper from Week 1 – CP-SAT hid what happens after that
- No solver reads a `.mzn` file directly: it is first flattened into FlatZinc

Definition: FlatZinc (`.fzn`)

The low-level, solver-agnostic intermediate format that every MiniZinc-compatible solver actually reads; produced by flattening a `.mzn` model.

Flattening: Three Concrete Changes

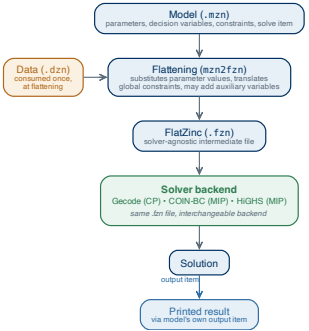
- Every parameter's value is substituted directly – n disappears, replaced by the number from the `.dzn` file
- Every global constraint (`all_different(q)`) maps to whichever FlatZinc predicate the target solver's library defines
- The flattener may introduce auxiliary variables of its own – Week 1's concept, now done by a compiler

Analogy: A Travel Agent's Booking Codes

Analogy

A travel agent converts a plain-language itinerary into a specific airline's own booking codes – the itinerary you wrote never changes, but each airline's system needs its own coded format. A MiniZinc model is translated into whichever FlatZinc predicates the chosen solver's library understands, without you rewriting anything by hand.

The Pipeline, End to End



`model.mzn` + `data.dzn` → flattening → `model.fzn` → solver → solution, printed back through the model's own `output` item.

A Version Detail: `include` Is Not Optional

Common Pitfall

In MiniZinc 2.10.1, `all_different` is no longer auto-included. A model calling it must start with `include "all_different.mzn";` – omit it, and `mzn2fzn` reports an undefined identifier before the model ever reaches a solver. MiniZinc's global constraints live in a standard library, not built in like CP-SAT's `AllDifferent`.

Both `nqueens.mzn` and `assignment.mzn` carry this line as their first non-comment statement.

Worked Example: N-Queens, Rewritten in MiniZinc

The same Week 1–4 N-queens model, in MiniZinc syntax:

```
int: n;
array[1..n] of var 1..n: q;
```

```
constraint all_different(q);
constraint all_different([q[i] + i | i in 1..n]);
constraint all_different([q[i] - i | i in 1..n]);
```

Three `all_different` calls – columns, then both diagonal transformations $q_i + i$ and $q_i - i$, unchanged from Week 1 Unit 3.

Reading a Flattened `.fzn` Excerpt

Compiled with `minizinc --compile -o nqueens.fzn nqueens.mzn` (`n = 6`):

Expected Output

```
predicate fzn_all_different_int(array [int] of var int: xs);
array [1..6] of var int: q :: output_array([1..6]) = [...];
array [1..6] of var int: X_INTRODUCED_13_ :: var_is_introduced = [...];
...
constraint fzn_all_different_int(q);
constraint fzn_all_different_int(X_INTRODUCED_13_);
...
solve  satisfy;
```

n is gone; `all_different` became `fzn_all_different_int`; `X_INTRODUCED_13_` is an auxiliary variable the flattener introduced.

3 • Participation Quick Check: Find the Predicate

Open the compiled `.fzn` file. Which FlatZinc line corresponds to `all_different(q)` on the columns, and does the literal string `all_different` still appear?

`constraint fzn_all_different_int(q);` – the literal identifier `all_different` does not survive flattening; it is translated to whichever FlatZinc predicate name the target solver's own library defines for that global constraint.

SECTION 3

Unit 3 — Gecode and a MIP Backend, Unchanged

2 • Content Two Students, One File, Two Solvers

Two students each own the exact same `assignment.mzn` file.

- One types `minizinc --solver Gecode assignment.mzn`
- The other types `minizinc --solver COIN-BC assignment.mzn`
- Neither edited a single line – one program now does backtracking search with propagation, the other solves a sequence of linear programs with branch-and-bound

Solver Independence, Precisely

Definition: Solver independence

The property that one MiniZinc model (source unchanged) can be solved by more than one solver backend, all agreeing on feasibility and, for optimization problems, on the optimal objective value.

- Guaranteed: the optimal objective value, feasibility/infeasibility
- Not guaranteed: solving time, which optimal solution is reported if several tie, internal search behaviour

Worked Example: The Assignment Problem

n workers, n tasks, an $n \times n$ cost matrix; assign each worker exactly one task, minimizing total cost.

```
array[1..n] of var 1..n: assign;
```

```
constraint all_different(assign);
```

Chosen over N-queens here precisely because it has an objective – “the optimal value matches” becomes a checkable claim.

This Environment's Real Solver Set

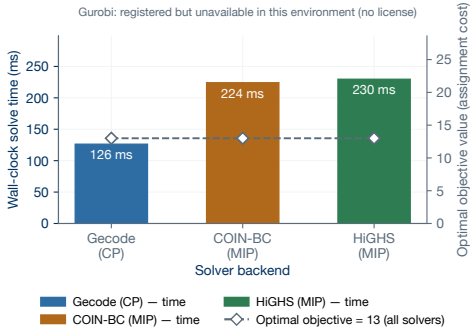
- `minizinc --solvers` lists Gecode plus three MIP *registrations* – Gurobi, HiGHS, COIN-BC
- Only **Gecode, HiGHS and COIN-BC** actually solve here
- Gurobi's MiniZinc plugin fails outright:

Expected Output

```
Error: MIPGurobiWrapper runtime error:
No Gurobi license found (user tolgaberber, ...)
```

Listed by `minizinc --solvers` and actually usable are two different claims – Chuffed and an OR-Tools FlatZinc target are not installed at all.

Real Data: Runtime Across Three Solvers



Same 4×4 assignment instance, optimal cost 13 on all three – Gecode 126 ms, COIN-BC 224 ms, HiGHS 230 ms. Gurobi is not plotted; it is recorded as unavailable.

3 • Participation Quick Check: Predict the Match

Gecode reports optimal cost 13 on a 4×4 assignment instance. You run the identical `.mzn`/`.dzn` pair on HiGHS. What value do you expect, and what might differ?

The optimal cost must also be 13 – one well-defined optimum regardless of which correct algorithm finds it. What may differ: which optimal assignment is reported if several tie, and how long each solver took.

SECTION 4

Unit 4 — MiniZinc from Python; Choosing a Solver

2 • Content Why Script This At All

- Retyping `minizinc --solver ...` three times and copying numbers into a spreadsheet does not scale
- It gives you no automatic way to notice, before wasting time on an error message, that a solver name is registered but will not run
- This unit gets Python to do the retyping, the copying, and the “does this solver actually work” check

The `minizinc` Package's Core Objects

- `Model` – loads a `.mzn` file
- `Instance` – a `Model` bound to a chosen `Solver`
- `Solver.lookup("gecode")` – finds an installed solver by the name `minizinc --solvers` reports
- `instance.solve()` returns a `Result`, whose `.solution` exposes decision-variable values – the same role `solver.Value(var)` played for CP-SAT

Code: Loading a Model and a Gecode Instance

```python
knapsack_model = minizinc.Model(MZN_DIR / "knapsack.mzn")
knapsack_model.add_file(MZN_DIR / "knapsack.dzn")
nqueens_model = minizinc.Model(MZN_DIR / "nqueens.mzn")
nqueens_model.add_file(MZN_DIR / "nqueens.dzn")
```

```python
solver = minizinc.Solver.lookup("gecode")
instance = minizinc.Instance(solver, model)
start = time.perf_counter()
result = instance.solve()
elapsed_ms = (time.perf_counter() - start) * 1000
rows.append(("Gecode", str(result.status), elapsed_ms, summarize(
    result)))
```

Code: Probing Each MIP Backend Before Relying On It

```python
for backend in MIP_BACKENDS:
    try:
        solver = minizinc.Solver.lookup(backend)
        instance = minizinc.Instance(solver, model)
        start = time.perf_counter()
        result = instance.solve()
        elapsed_ms = (time.perf_counter() - start) * 1000
        rows.append((backend, str(result.status), elapsed_ms, summarize(
            result)))
    except minizinc.MiniZincError as error:
        rows.append((backend, "unavailable", 0.0, str(error).splitlines()
            [0]))
```

A failed lookup or solve raises `minizinc.MiniZincError` – caught and recorded, not left to crash the script.

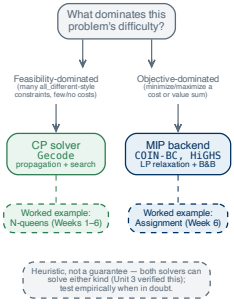
Output: The Comparison Table, For Real

Expected Output

```
knapsack (n=4, capacity=7):
solver     status             time_ms  summary
Gecode     OPTIMAL_SOLUTION    117.83  objective=9
coin-bc    OPTIMAL_SOLUTION    220.22  objective=9
highs      OPTIMAL_SOLUTION    221.70  objective=9
gurobi     unavailable           0.00  No Gurobi license found ...
```

Same objective, three real solvers – Gurobi reported as unavailable with its captured reason, not a crash.

Choosing Which Solver to Run



Feasibility-dominated, combinatorial problems (N-queens) tend to suit Gecode; objective-dominated, largely linear problems (assignment) tend to suit a MIP backend – a starting heuristic to test, not a theorem.

Analogy: A Bike Courier vs a Freight-Routing Service

Analogy

A bike courier is fast and flexible for a single point-to-point job with many local restrictions – a combinatorial, constraint-heavy job, like N-queens. A freight-routing service optimizes a numeric cost across many possible routes – a linear-objective job, like assignment. Either can deliver either package, but each is naturally suited to a different kind of job.

3 • Participation Quick Check: N-Queens or Assignment?

Using the decision guide's two structural signals, would you expect N-queens or the assignment problem to be the more natural fit for Gecode, and which for a MIP backend?

N-queens: pure satisfaction, combinatorial `all_different` constraints, no linear objective – Gecode's strength. Assignment: a simple feasibility constraint but a linear cost objective – exactly what LP relaxation and branch-and-bound target. Both solvers can, and in Unit 3 did, solve either correctly.

SECTION 5

Summary

Summary

Summary

- A `.mzn` model states parameters, decision variables, constraints and a `solve` item – no search logic, no host program
- Flattening (`mzn2fzn`) substitutes parameters, translates global constraints into FlatZinc predicates, and may add auxiliary variables – every solver reads this same format
- Solver independence guarantees the optimal value and feasibility across solvers – not timing, not which tied solution is reported
- The `minizinc` Python package scripts the exact same comparison across Gecode and a MIP backend without leaving Python

4 • Assessment Check Yourself

- One Check Yourself question closes each unit in this week's chapter – answers are collected at the chapter's end
- Parameter vs. decision variable in knapsack, why `all_different` does not survive flattening, what a Gecode/HiGHS run guarantees, Gecode or MIP for large bin-packing

5 • Follow-through Transfer — Where Else?

- Week 7 opens the box you never opened this week: you write LP/MIP formulations by hand and build `gurobipy` models directly for the same knapsack and assignment problems
- “Same model, different backend, same optimal value” is the habit Weeks 10–15 keep using to move between CP-SAT-style and MIP-style views of a problem without re-deriving it from scratch
- This course has no running capstone project – the thread is the *same* classic problems, revisited with new machinery

In the Lab

This week's lab sheet

You install MiniZinc and HiGHS yourself, confirm exactly which registered solvers actually solve, build and run all three models on Gecode, then on COIN-BC/HiGHS unchanged, run the Python comparison script, and close by building a complete MiniZinc model from scratch for a problem you have not seen modeled before.

- Next week: `gurobipy` – LP relaxation, branch-and-bound, and your first hand-built mathematical programs
- Keep `knapsack.mzn`, `nqueens.mzn` and `assignment.mzn` – Week 7 reuses their exact cost data